

Custom SIEM Correlation Rules for Credential Stuffing, DNS Tunnelling, and Po

1. Overview

This report shows how to design, implement, and test custom correlation rules in a SIEM, using an ELK-based stack as the working example. The focus is on three attack techniques that are common, high-signal when telemetry exists, and often noisy if rules are tuned poorly:

- Credential stuffing
- DNS tunnelling
- PowerShell exploitation

A correlation rule is not just a single keyword search. In a SIEM context, it usually means a query or pipeline that watches for related events over time and raises an alert when the pattern suggests an attack, not just a routine operation. The best rules combine good log sources, enough context to distinguish legitimate use from abuse, thresholds or sequencing that reduce false positives, and validation with realistic test data before production use.

This report is structured as a practical internship deliverable. It explains the methodology first, then walks through each attack technique with rule intent, data requirements, example detection logic, tuning considerations, and testing guidance.

2. General Rule Development Process

2.1 Start with the technique, not the query

For each technique, define what the attacker is trying to achieve, which systems see the activity, what normal behavior looks like, what abusive behavior looks like, which logs are required to see it, and what could create false positives. This keeps the rule tied to the actual attack rather than to a convenient log field.

2.2 Confirm the required telemetry

Correlation rules only work if the SIEM can see the activity. For the three techniques in this report, useful telemetry typically includes:

- Authentication logs, application login logs, SSO/IdP logs
- DNS query logs
- Network proxy or firewall logs
- Endpoint process telemetry, especially command-line and script execution events
- EDR or shell-monitoring data where available
- Application or API request logs for login endpoints and rate-limit events

If a required source is missing, the first deliverable should be a telemetry gap note, not a rule that cannot be validated.

2.3 Choose the rule type

In an ELK-style environment, detection can be implemented in several ways:

- Simple event-based rules for obvious patterns
- Correlation or sequence rules for multi-event behaviors
- Aggregation rules for volume-based detection, such as repeated failed logins
- Anomaly or threshold-based rules for behavior that deviates from baseline

The right type depends on the technique. Credential stuffing is often a volume and pattern problem, while PowerShell

exploitation may require sequencing of execution, command-line context, and follow-on actions.

2.4 Write the rule logic

A good rule definition should state the name and technique it addresses, the log sources used, the event criteria, the time window, the threshold or sequence conditions, the expected alert context, and tuning notes. If possible, map the rule to a known technique taxonomy such as MITRE ATT&CK so the detection has a clear security meaning.

2.5 Test before deployment

Testing should cover positive cases that should trigger the rule, negative cases that should not, edge cases such as bursts, retries, legitimate automation, and known benign tools, and sensitivity checks at different thresholds or windows. Testing is not complete until you know both how the rule fires and how often it might fire incorrectly.

2.6 Tune and document

After testing, adjust thresholds, lookback windows, the scope of source IPs, users, or destinations, and exclusion logic for known-good activity. Document the expected false-positive profile and the kind of investigation each alert should trigger.

3. Credential Stuffing

3.1 What the technique looks like

Credential stuffing is the use of large numbers of stolen username and password pairs against login interfaces. The hallmark is not a single failed login, but many authentication attempts across many accounts, often from a small number of sources or with shared tooling characteristics.

Typical indicators include:

- High rates of failed logins
- Many distinct accounts targeted from the same source or in the same time window
- Logins using credentials that appear in known breached datasets, if that data is available
- Unusual geographic or device patterns compared with the user's normal access
- Successful logins following many failures, especially from new locations or devices

3.2 Required log sources

Strong detection usually needs:

- Authentication failure and success events
- Source IP, user agent, and timestamp
- Account or username identity
- Session or device context where available
- Rate-limiting or abuse-mitigation logs from the application or SSO layer

If only failed-login counts are available, detection is still possible but weaker and noisier.

3.3 Rule intent

The rule should surface sequences that look like automated reuse of credentials rather than a user who simply forgot a password. A strong rule should distinguish one user with several failures, which is usually not stuffing, from many users targeted from the same source in a short period, which is higher concern. High failure volume followed by a success is

also higher concern, as is login attempts with randomised or suspicious client behavior.

3.4 Example detection logic

A practical correlation approach can combine several signals: failure rate per source IP or subnet over a short window, the number of distinct accounts attempted from the same source, rapid repeated attempts with minimal delay between them, optional success shortly after many failures from the same source, and optional known malicious user-agent patterns.

```
RULE CONCEPT – CREDENTIAL STUFFING
WINDOW: 5 to 15 minutes
IF source_ip generates failed login attempts against many distinct accounts above a defined threshold
THEN raise alert
IF source_ip shows many failures and then a success for one of those accounts
THEN raise higher-severity alert
IF failure volume AND account diversity both exceed threshold
THEN include the combination in alert context
```

3.5 Tuning considerations

Credential stuffing rules can be noisy if they are too broad. Common tuning levers include:

- Thresholds for failed attempts and distinct accounts
- Time windows
- Whether to alert on a single IP or also on aggregated source groups
- Whether to require a success event for higher severity
- Exclusion of known authentication health checks or internal scanning where appropriate

Be careful not to tune away real attacks by requiring an unrealistically high threshold. The rule should be sensitive enough to catch automated campaigns, but specific enough to avoid alerting on a single confused user.

3.6 False-positive risks

Likely false-positive sources include:

- Users repeatedly entering the wrong password
- Automated synchronisation or API clients with stale credentials
- Password reset or account unlock activity
- Load testing or internal QA activity
- Misconfigured applications retrying authentication

These should be identified during testing so they can be excluded or deprioritised.

3.7 How to test it

Test the rule against sample data that represents both benign and malicious behavior.

Positive test cases:

- A source IP attempting logins against many different accounts in a short period
- A burst of failures followed by one success
- A pattern of attempts with consistent timing suggestive of automation

Negative test cases:

- A single user failing several times and then succeeding

- A small number of failures spread over a long period
- Legitimate password reset activity

Validation checklist:

- Does the rule fire for the simulated stuffing pattern?
- Does it stay silent for ordinary individual failures?
- Does the alert include the source IP, targeted accounts, time window, and failure count?
- Does severity change appropriately when a success follows many failures?
- Does the rule behave acceptably when volume is high but account diversity is low?

If possible, reproduce test events in a non-production environment and verify both the alert and the accompanying context.

4. DNS Tunnelling

4.1 What the technique looks like

DNS tunnelling uses DNS queries and responses to carry data or commands outside the intended channel. It is attractive to attackers because DNS is usually allowed and often less closely monitored than HTTP or other traffic.

Common signs include:

- Unusually large DNS queries or responses
- High volume of DNS queries from a single host
- Many queries with long or unusually encoded hostnames
- Repeated queries to the same uncommon domain or subdomain pattern
- Query patterns that look like encoded data rather than normal human-readable names
- Use of low-reputation or newly observed domains in an unusual way

4.2 Required log sources

Good detection depends on DNS logging that captures enough detail:

- Query timestamp
- Source host or IP
- Query name
- Query type
- Response details where available
- Domain or subdomain structure
- Optional: response size, flags, or resolution result

Network security logs can add context, such as which internal hosts are generating abnormal DNS traffic.

4.3 Rule intent

The rule should identify DNS usage that does not match normal host behavior and is consistent with covert data movement or command-and-control via DNS. Rather than relying on one magic pattern, the strongest approaches use a mix of volume anomalies, query length or entropy anomalies, unusual query name structure, repeated contact with suspicious domains, and deviation from a host's normal DNS behavior.

4.4 Example detection logic

One approach is an aggregation and anomaly rule. For each host, measure DNS query volume over a window, flag hosts whose query volume is much higher than expected, look for a high proportion of long, oddly structured, or highly varying subdomain names, and flag repeated queries to rare or suspicious domains.

Another approach is pattern-based:

- Alert on DNS queries where the query name is unusually long or contains high-entropy strings
- Alert when a single host generates many such queries in a short period
- Combine with reputation signals where available

```
RULE CONCEPT — DNS TUNNELLING
WINDOW: short-term host-level aggregation
IF a host generates an unusually high count of DNS queries with long or randomly appearing subdomains
THEN raise an alert
IF those queries concentrate on one domain that is rare in the environment
THEN increase severity
IF the host also shows other suspicious behavior
THEN correlate that into a broader investigation alert
```

4.5 Tuning considerations

DNS rules must be tuned carefully because legitimate services can sometimes look unusual.

Useful tuning factors:

- Baseline DNS behavior per host or per subnet
- Normal domains used by the organization
- Known internal or vendor domains that generate many queries
- Time window and volume thresholds
- Minimum query name length or entropy threshold, if used
- Whether to alert on first occurrence or require repetition

A rule that fires on any long DNS name will likely create too many false positives. It is usually better to require both unusual structure and unusual volume or repetition.

4.6 False-positive risks

Common sources of noise include:

- Software updates and package registries
- Telemetry or analytics services
- CDNs and dynamic resource lookups
- VPN or productivity tools that use many subdomains
- Legitimate internal services with automated name generation

During testing, identify which normal services create the busiest or longest query patterns so they can be baselined or excluded where appropriate.

4.7 How to test it

Positive test cases:

- A host generating many high-entropy or long DNS queries in a short period

- Repeated queries to a single uncommon domain with randomised subdomains
- A spike in DNS traffic from a host that normally has low DNS volume

Negative test cases:

- Normal browsing and application activity with ordinary DNS patterns
- Periodic but expected lookups to known services
- A single unusual DNS query without repetition or volume

Validation checklist:

- Does the rule fire only when both anomaly and repetition signals are present?
- Does it avoid flagging a single odd-looking query?
- Does the alert include source host, domain, query count, and time window?
- Can you distinguish suspicious tunneling-like patterns from a legitimate high-volume DNS client?
- Does the rule hold up when tested against realistic internal traffic patterns?

If DNS logs are available in a non-production environment, replay or construct representative events to check both detection and noise levels.

5. PowerShell Exploitation

5.1 What the technique looks like

PowerShell is a powerful administration tool, which also makes it useful to attackers for execution, discovery, downloading payloads, and lateral movement. The risk is not PowerShell itself, but how it is used.

Suspicious use often includes:

- PowerShell invoked from unexpected parent processes
- Command lines that download or decode content
- Use of obfuscated or encoded commands
- Attempts to bypass constraints or hide activity
- PowerShell used to launch or inject into other processes
- Unusual scripting activity on workstations or servers that do not normally use PowerShell in that way

5.2 Required log sources

Better detection usually needs:

- Process creation events
- Parent process information
- Full or meaningful command-line capture
- User context
- Host and timestamp
- Script block logging or module logging where available
- EDR telemetry for follow-on behavior such as network connections, file writes, or injection

If command-line logging is limited, detection becomes harder and more reliant on process ancestry and outcomes.

5.3 Rule intent

The rule should highlight PowerShell usage that is either intrinsically risky or inconsistent with the host's normal administration patterns. Detection can be built around high-risk command-line features or arguments, suspicious parent-child relationships, execution of downloaded or encoded content, scripting behavior on systems that rarely use PowerShell, and sequences where PowerShell execution is followed by further suspicious actions.

5.4 Example detection logic

A strong approach uses layered rules rather than one giant pattern.

Layer 1: high-risk command-line indicators

- Alert on PowerShell commands that include download or request features, decoding features, execution features, or other behaviors associated with payload handling
- Alert on heavily obfuscated command lines, especially when combined with encoded commands

Layer 2: context-based correlation

- Alert when PowerShell is spawned by an unexpected parent process, such as a browser, office application, or other user-facing process where scripting is not normally expected
- Alert when PowerShell use occurs on a host or by a user outside normal patterns

Layer 3: sequence-based correlation

- If PowerShell execution is followed by suspicious network connections, file writes to unusual locations, or process creation patterns of concern, raise a higher-severity alert

RULE CONCEPT – POWERSHELL EXPLOITATION

LAYER 1: Flag PowerShell with encoded/obfuscated arguments or features commonly used to download or run additional content

LAYER 2: If that execution comes from a suspicious parent or an unusual host context, escalate

LAYER 3: If additional suspicious actions follow within a short window, correlate them into a possible exploitation event

5.5 Tuning considerations

PowerShell rules can easily become too noisy if they rely on presence alone. Focus on the most suspicious command-line features first, use parent-process and host context to reduce benign administrative noise, use allowlists carefully and only for well-understood verifiable use cases, prefer correlation over single-event alerting where possible, and keep the rule tied to behaviors that are hard to explain as normal administration. If in doubt, lower severity rather than silently disabling detection.

5.6 False-positive risks

Typical false positives include:

- Legitimate administrative scripts
- Software installation or management tools that use PowerShell
- Automated configuration management
- Helpdesk or support activity
- Developers or power users running scripts locally

These should be mapped during testing so the rule can distinguish everyday use from abuse.

5.7 How to test it

Positive test cases:

- PowerShell launched from an unexpected parent process with suspicious command-line features
- Encoded or obfuscated command execution
- PowerShell activity followed by a suspicious network connection or file operation in the same window

Negative test cases:

- Normal admin script execution in an expected context
- Routine software deployment or configuration activity
- Low-risk PowerShell commands without suspicious features

Validation checklist:

- Does the rule fire for risky PowerShell usage but not for routine admin scripts?
- Does parent-process and host context change the alert correctly?
- Does the alert provide enough detail to justify investigation, including command line, parent process, user, and host?
- Does sequence correlation improve confidence without hiding suspicious activity?
- Does the rule avoid flagging every PowerShell execution in the environment?

Because PowerShell detection depends heavily on logging quality, testing should also confirm that the necessary fields are actually being captured and searchable.

6. Testing and Validation Strategy

6.1 Define test objectives

For each rule, decide what a successful test means. The rule should detect the intended technique, should not drown analysts in false alerts, should provide actionable context, and should behave predictably when conditions change.

6.2 Build representative test data

Use a mix of simulated attack events, normal activity events, and edge cases such as retries, bursts, and partial patterns. If possible, create test events that are as close as possible to the real log schema used in the SIEM. Testing with the wrong field names or formats can hide problems until deployment.

6.3 Use a structured validation approach

For each rule, validate:

- Detection: does it fire for the intended pattern?
- Specificity: does it avoid firing for obvious benign cases?
- Context: does the alert contain the fields needed to triage it?
- Threshold behavior: does severity or alert frequency change in sensible ways?
- Robustness: does the rule still make sense under higher event volumes?

6.4 Review false positives and missed detections

After testing, classify outcomes as true positive, false positive, false negative, or low-confidence case. Use these results to adjust thresholds, add context, or improve log coverage.

6.5 Operationalize carefully

Before production use, document the rule's purpose, data dependencies, and known limitations, set alert severity and

routing, define initial response steps, identify who owns tuning and who reviews alerts, and plan for periodic review so rules stay relevant as systems and attackers change.

7. Suggested Rule Summary

7.1 Credential stuffing

- Detect high volumes of authentication failures across many accounts from shared sources
- Escalate when failures are followed by a success
- Tune by threshold, window, account diversity, and client behavior
- Validate against both individual user mistakes and automated retry patterns

7.2 DNS tunnelling

- Detect abnormal DNS volume and unusual query name structure
- Combine anomaly signals with repetition or rare-domain usage
- Baseline normal host and service behavior before raising alerts
- Validate against legitimate high-volume DNS clients and single odd queries

7.3 PowerShell exploitation

- Detect high-risk PowerShell command-line patterns and suspicious process ancestry
- Correlate with host context and follow-on suspicious actions
- Use layered rules to reduce noise while preserving visibility
- Validate against both malicious usage and legitimate administrative scripts

8. Conclusion

Custom correlation rules work best when they are built around the attack technique, supported by the right logs, and tested against realistic behavior before deployment. Credential stuffing, DNS tunnelling, and PowerShell exploitation each require different detection strategies: volume and pattern analysis for credential stuffing, anomaly and structure analysis for DNS tunnelling, and command-line, parent-process, and sequence analysis for PowerShell exploitation.

The most common failure mode is not a lack of rule ideas, but weak telemetry, poor thresholds, or insufficient testing. A repeatable process of requirements, log coverage, rule logic, test data, tuning, and validation will produce rules that are far more useful in practice than one-off queries alone.

For an internship deliverable, the strongest submission pairs the rule concept with proof that it was tested, explains what the rule catches, what it misses, and what would be needed to improve it. That shows both detection thinking and operational maturity.